

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



یادگیری ماشین

تمرین شماره ۲

نام و نام خانوادگی
مجید صادقی نژاد

شماره دانشجویی
۸۱۰۱۰۱۴۵۹

۴ آذر ۱۴۰۴

۲	۱ پرسش ۱ – Line Search Optimization Problem	۱
۲	General form and roles	۱.۱
۲	Descent direction - definition and necessary condition	۲.۱
۳	Common methods to choose the step length	۳.۱
۳	Specific quadratic example	۴.۱
۵	۲ سوال ۳ – Bilevel Optimization	۲
۵	Bilevel formulation (upper and lower)	۱.۲
۵	Solve the follower for fixed (x)	۲.۲
۶	Substitute ($y^*(x)$) into leader objective	۳.۲
۶	Find the optimal (x^*, y^*)	۴.۲
۷	Regularity, convexity, uniqueness and substitution validity	۵.۲
۸	۳ سوال ۵ – Genetic Algorithm - Jigsaw Puzzle Solver	۳
۸	Edge Descriptors and Fit Score	۱.۳
۹	Whole Puzzle Fitness	۲.۳
۹	Selection	۳.۳
۱۰	Evolutionary Engine	۴.۳
۱۱	۴ سوال ۷ – Batch vs. Stochastic Gradient Descent	۴
۱۲	Computational load per iteration	۱.۴
۱۲	Noise (variance) in the gradient estimation	۲.۴
۱۲	Stability and nature of convergence	۳.۴
۱۳	۵ سوال ۸ – Steepest Descent Direction via First-Order Taylor	۵

۱۵	Quadratic Convergence of Newton's Method – سوال ۱۱		۶
۱۵		quadratic convergence and linear convergence	۱.۶
۱۵	. .	The Newton direction approximates the exact displacement $x^* - x_k$	۲.۶
۱۸	Rosenbrock Function: GD vs. Newton – سوال ۱۲		۷
۱۸		Gradient and Hessian	۱.۷
۱۸		Implementation	۲.۷
۱۹		Parameters used	۳.۷
۱۹		Numerical results	۴.۷
۲۰		Plots and discussion of convergence rates	۵.۷
۲۲	Step-Size Strategies in Gradient Descent – سوال ۱۴		۸
۲۲		Two Step-Size Selection Methods	۱.۸
۲۲		Implementation on the Rosenbrock Problem	۲.۸
۲۳		Comparison of Results	۳.۸

فهرست تصاویر

۱۱		Result of solving puzzle	۱
۱۱		Progress of GA	۲
۲۰		Plots of Convergence rate	۳
۲۴		Trajectories for Armijo GD and AdaGrad	۴

فهرست جداول

۱۹	 Different Approches Numerical Results	۱
----	---	---

In this homework, we explored how optimization methods improve the process of finding the best solution in mathematical problems. We experimented with gradient-based algorithms and saw how different step-size strategies can affect speed and accuracy. Using the Rosenbrock function as a case study, we observed how algorithms move through challenging landscapes. We compared traditional methods with adaptive ones and saw their strengths and weaknesses in practice. Overall, the project showed how choosing the right optimization strategy can greatly influence convergence and performance.

پرسی ۱ - Line Search Optimization Problem

۱.۱ General form and roles

General problem solved by line-search algorithms:

$$\min_{x \in \mathbb{R}^n} f(x)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable.

A line-search algorithm at iteration (t) picks: a search direction $p^{(t)}$ (a vector in $\mathbb{R}^n$) and a step length (scalar) ($\alpha^{(t)} > 0$) and updates

$$x^{(t+1)} = x^{(t)} + \alpha^{(t)} p^{(t)}$$

Search direction p : chooses where to move from the current point (steepest descent uses $p = -\nabla f$).

Step length (α): decides how far to move along that direction.

۲.۱ Descent direction - definition and necessary condition

A vector p is a descent direction at point x if moving a small positive amount along p decreases the function value

$$f(x + \alpha p) < f(x)$$

A necessary condition for p to be a descent direction at x is:

$$\nabla f(x)^\top p < 0$$

Geometric intuition: the directional derivative in direction p is negative.

Common methods to choose the step length ३.१

- Exact line search:

Choose $(\alpha^* = \arg \min_{\alpha > 0} f(x + \alpha p))$. Gives the best possible decrease along the line, but often expensive or impossible in closed form.

- Armijo / Backtracking:

Pick an initial α ($\alpha_0 = 1$) and reduce it (multiply by $\beta \in (0, 1)$) until the Armijo (sufficient decrease) condition holds:

$$f(x + \alpha p) \leq f(x) + c_1 \alpha \nabla f(x)^\top p$$

with small constant $c_1 \in (0, 1)$. it is Cheap and robust; ensures a sufficient decrease.

- Wolfe conditions:

Two conditions: Armijo (sufficient decrease) and a curvature condition guaranteeing $(|\nabla f(x + \alpha p)^\top p|)$ is small enough. Wolfe ensures both sufficient decrease and that (α) is not too small (keeps reasonable curvature). Strong Wolfe replaces curvature by $(|\cdot|)$ bound.

- Fixed step

Use a constant α . Simple but may be inefficient or unstable.

- Barzilai-Borwein

Not a true line search — sets step length using quasi-Newton-like formulae computed from previous iterates (uses differences $(s^{(t)} = x^{(t)} - x^{(t-1)})$ and $(y^{(t)} = \nabla f^{(t)} - \nabla f^{(t-1)})$). Often accelerates gradient methods in practice.

Specific quadratic example ४.१

Function:

$$f(x_1, x_2) = 2x_1^2 + x_1x_2 + 4x_2^2$$

At iteration (t):

$$x^{(t)} = (0, 1) \quad p^{(t)} = (1, -2)$$

- Compute gradient ∇f . For $f(x_1, x_2) = 2x_1^2 + x_1x_2 + 4x_2^2$

$$\frac{\partial f}{\partial x_1} = 4x_1 + x_2, \quad \frac{\partial f}{\partial x_2} = x_1 + 8x_2$$

Evaluate at $x^{(t)} = (0, 1)$:

$$\frac{\partial f}{\partial x_1} = 4 \cdot 0 + 1 = 1 \quad \frac{\partial f}{\partial x_2} = 0 + 8 \cdot 1 = 8$$

So $\nabla f(x^{(t)}) = (1, 8)$, Now dot with $p^{(t)} = (1, -2)$:

$$\nabla f(x^{(t)})^\top p^{(t)} = 1 \cdot 1 + 8 \cdot (-2) = 1 - 16 = -15$$

Since $(-15 < 0)$, $p^{(t)}$ is a descent direction at $x^{(t)}$.

- Compute

$$\begin{aligned} \phi(\alpha) &= f(x^{(t)} + \alpha p^{(t)}) \\ &= f(\alpha, 1 - 2\alpha) \\ &= 2(\alpha)^2 + (\alpha)(1 - 2\alpha) + 4(1 - 2\alpha)^2 \\ &= 16\alpha^2 - 15\alpha + 4 \end{aligned}$$

- For quadratic ($\phi(\alpha) = a\alpha^2 + b\alpha + c$), the minimizer is

$$\alpha^* = -\frac{b}{2a}$$

Here ($a = 16$, $b = -15$). So

$$\alpha^* = \frac{15}{32}$$

Also check second derivative: ($\phi''(\alpha) = 32 > 0$), so this is indeed a minimum.

Bilevel Optimization – سوال ۳

Leader:

$$\min_{x \in \mathbb{R}} F(x, y) = (x - 2)^2 + (y - 3)^2 \quad x \geq 0$$

Follower (given (x)):

$$\min_{y \in \mathbb{R}} f(x, y) = (y - x^2)^2 + y \quad y \geq 0$$

Bilevel formulation (upper and lower) ۱.۲

Upper (leader):

$$\min_{x \geq 0} F(x, y^*(x))$$

where $(y^*(x))$ is the follower's response.

Lower (follower): for given (x)

$$y^*(x) = \arg \min_{y \geq 0} (y - x^2)^2 + y$$

Solve the follower for fixed (x) ۲.۲

Consider $(g_x(y) = (y - x^2)^2 + y)$ with constraint $(y \geq 0)$.

$$g'_x(y) = 2(y - x^2) + 1 = 2y - 2x^2 + 1 = 0$$

$$\Rightarrow y = x^2 - \frac{1}{2}$$

Because $(g''_x(y) = 2 > 0)$, the function is strictly convex in y . So the unconstrained minimizer is unique. Enforce the constraint $(y \geq 0)$. Therefore:

$$y^*(x) = \max\left(0, x^2 - \frac{1}{2}\right)$$

Substitute $(y^*(x))$ into leader objective ۳.۲

Define $F(x) = F(x, y^*(x))$, $(x \geq 0)$.

Case 1: $(x^2 \leq \frac{1}{2})$. Then $(y^* = 0)$ and

$$F(x) = (x - 2)^2 + (0 - 3)^2 = (x - 2)^2 + 9$$

Case 2: $(x^2 > \frac{1}{2})$. Then $(y^* = x^2 - \frac{1}{2})$ and

$$F(x) = (x - 2)^2 + (x^2 - \frac{1}{2} - 3)^2 = (x - 2)^2 + (x^2 - 3.5)^2$$

So the bilevel reduces to minimizing $(F_{\text{red}}(x))$ over $(x \geq 0)$, piecewise as above.

Find the optimal (x^*, y^*) ۴.۲

Check Case 1 $(0 \leq x \leq \frac{1}{\sqrt{2}})$:

$$F(x) = (x - 2)^2 + 9$$

This is decreasing on this interval (derivative $(2(x - 2) < 0)$ since $(x \leq \frac{1}{\sqrt{2}} < 2)$), so the minimum on this interval is attained at the right endpoint $(x = \frac{1}{\sqrt{2}})$. Its value:

$$F\left(\frac{1}{\sqrt{2}}\right) \approx 10.672$$

Check Case 2 $(x > \frac{1}{\sqrt{2}})$:

$$F(x) = (x - 2)^2 + (x^2 - 3.5)^2$$

$$\frac{dF}{dx} = 2(x - 2) + 4x(x^2 - 3.5) = 4x^3 - 12x - 4 = 0$$

$$\Rightarrow x^3 - 3x - 1 = 0$$

This cubic has one real root $(x \approx 1.879)$. At this (x),

$$y^* = x^2 - \frac{1}{2} \approx 3.032$$

Evaluate the leader objective there:

$$F(x) \approx 0.016$$

which is much smaller than the Case 1 minimum (10.67). Thus the global minimizer of the bilevel problem is

$x^* \approx 1.879 \quad y^* \approx 3.032$

Regularity, convexity, uniqueness and substitution validity 5.2

- Convexity of the follower problem: For fixed (x) , the follower objective ($g_x(y) = (y - x^2)^2 + y$) has second derivative ($g_x''(y) = 2 > 0$). So it is strictly convex in (y) . With the convex constraint ($y \geq 0$), the follower problem is convex and has a unique minimizer for every x .
- Uniqueness: Because of strict convexity, the argmin ($y^*(x)$) is unique for every (x) . When the unconstrained minimizer ($x^2 - \frac{1}{2}$) is negative, the unique constrained minimizer is the boundary point ($y = 0$). So uniqueness holds everywhere.
- Validity of substitution: Since the follower response ($y^*(x)$) is single-valued for every (x) , the bilevel problem can be reduced to a single-level problem by substituting ($y^*(x)$) into the leader objective. That is what we did.
- Differentiability, regularity caveat: The mapping ($y^*(x) = \max(0, x^2 - \frac{1}{2})$) is continuous but not differentiable at the switching points where ($x^2 = \frac{1}{2}$) (i.e. ($x = \pm 1/\sqrt{2}$); only ($x = +1/\sqrt{2}$) matters since ($x \geq 0$)). So the reduced leader objective ($F(x)$) is piecewise-smooth but not twice-differentiable at that switch. This means first-order optimality conditions derived under smoothness assumptions (e.g. differentiating a KKT system) must be used with care at switching points. However,

in this example the global minimizer lies in the smooth region ($x > 1/\sqrt{2}$), so standard differentiation and stationarity conditions applied there are valid.

- Conclusion on regularity: The follower's strict convexity guarantees a unique response and justifies substitution. Non-smoothness at the boundary is the only regularity issue to watch for; it can create corners where KKT-sensitivity arguments must be treated carefully.

Genetic Algorithm - Jigsaw Puzzle Solver – سوال ۵

Automatically assemble an image split into a grid of rectangular (or square) pieces using a Genetic Algorithm (GA). The code implements: (a) an edge fit score, (b) whole-puzzle fitness evaluation, (c) a selection mechanism favoring high-quality assemblies, (d) a full evolutionary loop. The source image is divided into a $R \times C$ grid. Each piece p has an ID and pixel array. Allowed rotations are $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ if pieces are square; otherwise only 0° . For every allowed orientation all four edge descriptors are precomputed.

Edge Descriptors and Fit Score ۱.۳

For an oriented piece edge we extract:

- Mean RGB color: $\mathbf{m} \in \mathbb{R}^3$.
- Downsampled edge color profile: $\mathbf{q} \in \mathbb{R}^{3L}$.

Given adjacent edges (a, b) the fit score is:

$$F(a, b) = \exp\left(-\alpha \|\mathbf{m}_a - \mathbf{m}_b\|\right) + \beta \cdot \frac{1 + \cos(\mathbf{q}_a, \mathbf{q}_b)}{2}$$

where $\cos(\mathbf{q}_a, \mathbf{q}_b) = \frac{\mathbf{q}_a^\top \mathbf{q}_b}{\|\mathbf{q}_a\| \|\mathbf{q}_b\|}$ and hyperparameters α and β weight color vs. profile similarity. The first term rewards low color distance; the second term rewards structural similarity along the edge. Higher F means a better match.

Whole Puzzle Fitness ۲.۳

Let permutation π map grid positions (r, c) to piece IDs, and let orientation array $o[\text{pid}]$ give the chosen rotation. The global fitness sums all horizontal and vertical adjacencies:

$$\text{Fitness}(\pi, o) = \sum_{r=0}^{R-1} \sum_{c=0}^{C-1} \left(\mathbf{1}_{c < C-1} F((\pi_{r,c}, o), (\pi_{r,c+1}, o)) + \mathbf{1}_{r < R-1} F((\pi_{r,c}, o), (\pi_{r+1,c}, o)) \right)$$

No penalties are added here; border constraints can be incorporated later. This scalar directly guides evolution: higher sums indicate more coherent assemblies.

Selection ۳.۳

Tournament selection of size k is used:

1. Sample k individuals uniformly from the population.
2. Select the one with highest fitness as a parent.

This balances exploitation (good solutions favored) and exploration (random sampling). Elitism preserves the top E individuals unchanged each generation.

Crossover: Order Crossover (OX) applied to the permutation maintains relative ordering and produces a valid permutation without duplicates. Piece orientations are inherited from the parent that contributes the piece segment; other orientations taken from the second parent.

Mutation:

- Swap two random positions (permutation).
- Local shuffle of a short contiguous segment.

- Random reorientation (if multiple orientations allowed) per piece with probability p_{rot} .

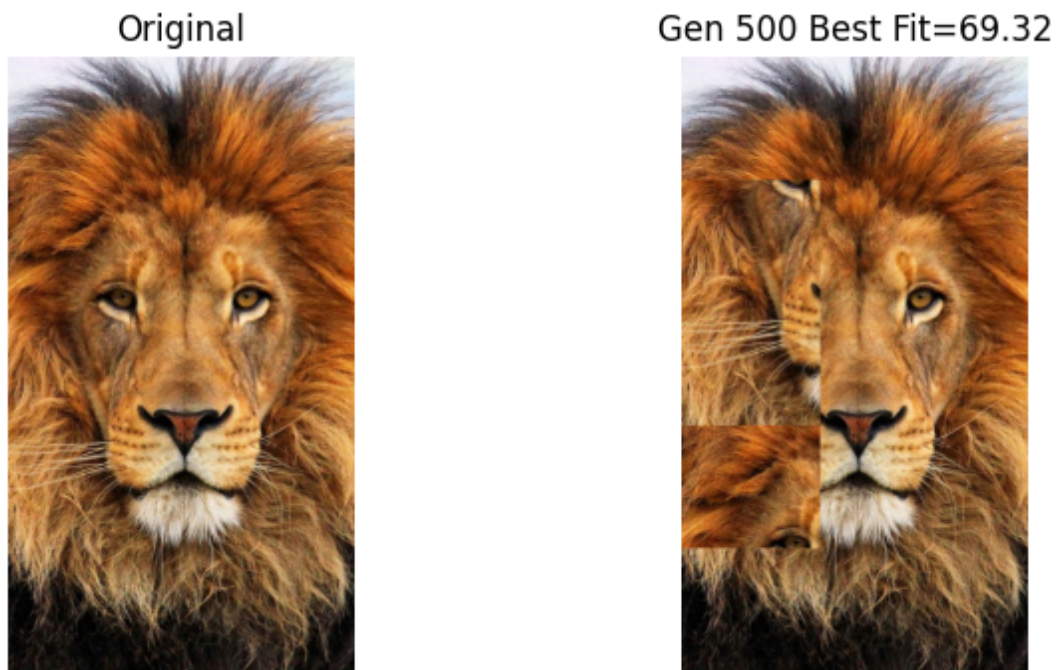
These mutations introduce diversity and permit escape from local optima.

Evolutionary Engine ۴.۳

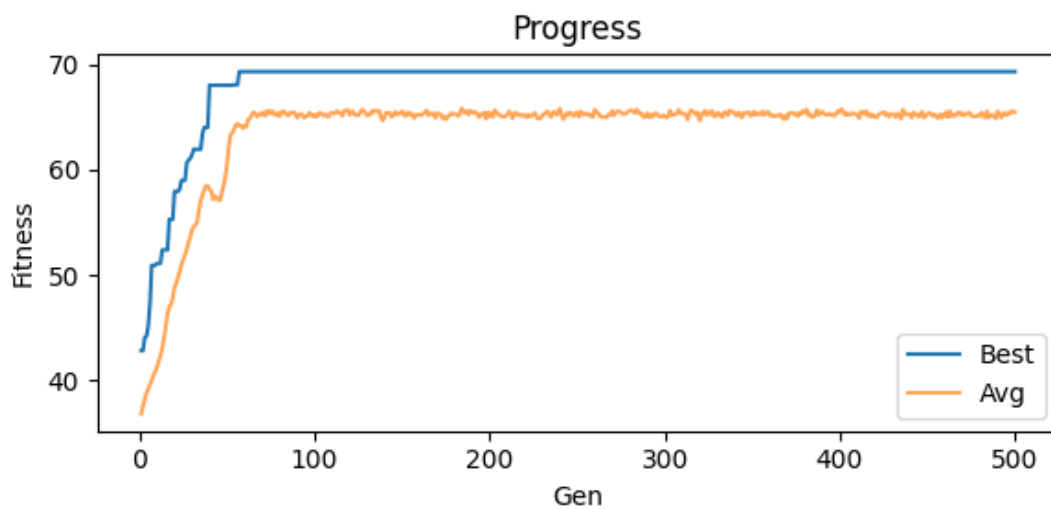
1. **Initialization:** Population of size P with random permutations and orientations.
2. **Evaluation:** Compute fitness for all individuals.
3. **Loop for G generations:**
 - Copy E elites.
 - While new population not full:
 - (a) Select parents via tournaments.
 - (b) Apply crossover with probability p_c .
 - (c) Apply mutation.
 - (d) Evaluate child; append.
 - Track best and average fitness.
 - (Optional) Visualize best arrangement each generation.
4. **Return** best individual found.

As results we can see early generations show random mosaics with low fitness; over time coherent regions form as matching edges accumulate. Best fitness typically plateaus after sufficient generations, indicating assembled contiguous areas of the original image. **Limitations and Extensions**

- Rectangular pieces lack true jigsaw tab/slot geometry; future work should integrate shape compatibility.
- No explicit border constraints; outer edges could be biased by penalizing mismatches or freezing orientations.



شکل ۱ : Result of solving puzzle



شکل ۲ : Progress of GA

سوال ۷ - Batch vs. Stochastic Gradient Descent

Computational load per iteration 1.4

Batch Gradient Descent (BGD): In BGD we compute the gradient using *all* training examples. If we have N data points, then each update requires processing all N points. Therefore, the cost per iteration is *high*:

$$\text{Cost per iteration (BGD)} = O(N).$$

Stochastic Gradient Descent (SGD): In SGD each update uses only *one* randomly chosen training example. So each update is very cheap:

$$\text{Cost per iteration (SGD)} = O(1).$$

BGD has heavy computations per iteration, while SGD has very light and fast updates.

Noise (variance) in the gradient estimation 2.4

Batch Gradient Descent (BGD): Since BGD uses the whole dataset, the gradient is the *exact* gradient of the loss. It has almost no noise or variance.

$$\widehat{\nabla f_{\text{BGD}}}(x) = \nabla f(x) \quad (\text{low variance}).$$

Stochastic Gradient Descent (SGD): SGD uses only one sample, so its gradient estimate is *noisy*. The direction is correct *on average*, but varies from step to step:

$$\mathbb{E}[\widehat{\nabla f_{\text{SGD}}}(x)] = \nabla f(x), \quad \text{but variance is high.}$$

BGD has stable, low-variance gradients. SGD has noisy, high-variance gradients.

Stability and nature of convergence 3.4

Batch Gradient Descent (BGD): Because BGD uses an exact gradient, its updates are *stable and smooth*. It moves steadily toward the minimum and typically converges reliably when

the step size is well chosen. It can reach very close to the true minimizer.

$$x_{t+1} = x_t - \alpha \nabla f(x_t) \quad (\text{stable path}).$$

Stochastic Gradient Descent (SGD): SGD moves in a noisy way toward the minimum. Its path “zig-zags,” especially when the step size is not small. With a fixed step size, SGD does not converge to the exact minimum but hovers around it. With a decreasing step size, it can converge, but more slowly.

$$x_{t+1} = x_t - \alpha_t \widehat{\nabla} f(x_t) \quad (\text{noisy path}).$$

- BGD: stable, smooth, can converge exactly.
- SGD: noisy, may only converge to a neighborhood unless step size decreases.

Steepest Descent Direction via First-Order Taylor – سوال ۸

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable and fix the current point x_k . We want to show that, in a small neighborhood of x_k , moving in the direction of the negative gradient gives the largest instantaneous decrease of f among all unit directions.

Step 1: First-order Taylor approximation. For any unit vector p (i.e. $\|p\| = 1$) and for small $\alpha > 0$ the first-order Taylor expansion gives

$$f(x_k + \alpha p) = f(x_k) + \alpha \nabla f(x_k)^\top p + o(\alpha),$$

where $o(\alpha)/\alpha \rightarrow 0$ as $\alpha \rightarrow 0^+$. For sufficiently small α the $o(\alpha)$ term is negligible compared to the linear term, so minimizing $f(x_k + \alpha p)$ over unit p is (to first order) equivalent

to minimizing the linear term $\nabla f(x_k)^\top p$.

Step 2: Minimization over unit vectors. We therefore consider the problem

$$\min_{\|p\|=1} \nabla f(x_k)^\top p.$$

By the Cauchy–Schwarz inequality,

$$\nabla f(x_k)^\top p \geq -\|\nabla f(x_k)\| \|p\| = -\|\nabla f(x_k)\|,$$

and equality is achieved exactly when p is collinear with $-\nabla f(x_k)$, i.e.

$$p = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|},$$

provided $\nabla f(x_k) \neq 0$. Thus the minimal possible value of $\nabla f(x_k)^\top p$ over unit p is $-\|\nabla f(x_k)\|$, attained by the negative normalized gradient.

Step 3: Conclusion. For small $\alpha > 0$, the direction $p = -\nabla f(x_k)/\|\nabla f(x_k)\|$ yields the largest first-order decrease

$$f(x_k + \alpha p) \approx f(x_k) - \alpha \|\nabla f(x_k)\|,$$

so the (unnormalized) steepest descent direction is $p_k = -\nabla f(x_k)$. If $\nabla f(x_k) = 0$ then every direction is (first-order) neutral and we are at a stationary point.

Remark. This argument relies on the first-order approximation: it shows that $-\nabla f(x_k)$ is the steepest descent direction in a local (infinitesimal) sense. For finite steps α higher-order terms matter, which is why practical algorithms also choose step lengths (line search) rather than taking a fixed large step.

سوال ۱۱ – Quadratic Convergence of Newton's Method

۱.۶ quadratic convergence and linear convergence

Let x^* be a (local) solution and define the error at iteration k by

$$e_k := x_k - x^*.$$

Quadratic convergence. An iterative sequence $\{x_k\}$ converges *quadratically* to x^* if there exists a constant $C > 0$ and an index K such that for all $k \geq K$

$$\|e_{k+1}\| \leq C\|e_k\|^2.$$

Equivalently, $\lim_{k \rightarrow \infty} \frac{\|e_{k+1}\|}{\|e_k\|^2} = L$ with $0 \leq L < \infty$, and in the ideal case $L = C$. The key feature is that the number of correct digits roughly doubles each iteration when $\|e_k\|$ is small enough.

Linear convergence. A sequence $\{x_k\}$ converges *linearly* to x^* if there exists $q \in (0, 1)$ and K such that for all $k \geq K$

$$\|e_{k+1}\| \leq q\|e_k\|.$$

Here the error is reduced by a constant factor each iteration; convergence is steady but much slower than quadratic. In particular, linear convergence implies $\|e_{k+1}\| = O(\|e_k\|)$, while quadratic gives $\|e_{k+1}\| = O(\|e_k\|^2)$, a much faster vanishing rate as $k \rightarrow \infty$.

۲.۶ The Newton direction approximates the exact displacement $x^* - x_k$

Assumptions. Assume $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is three times continuously differentiable in a neighborhood of a local minimizer x^* , with $\nabla f(x^*) = 0$ and the Hessian $H^* := \nabla^2 f(x^*)$ is

nonsingular (positive definite). Let $e_k = x_k - x^*$ be small.

Taylor expansion of the gradient. Expand ∇f about x^* . For $x_k = x^* + e_k$ we have the Taylor expansion

$$\nabla f(x_k) = \nabla f(x^*) + \nabla^2 f(x^*) e_k + r_k = H^* e_k + r_k,$$

where the remainder r_k satisfies (by the integral or Lagrange form of the remainder)

$$\|r_k\| = O(\|e_k\|^2) \quad \text{as } \|e_k\| \rightarrow 0.$$

(Concretely, $r_k = \frac{1}{2} \nabla^3 f(\xi_k)[e_k, e_k]$ for some ξ_k on the line segment between x^* and x_k .)

Hessian near the solution. Similarly expand the Hessian at x_k :

$$\nabla^2 f(x_k) = H^* + E_k, \quad \text{with } \|E_k\| = O(\|e_k\|).$$

Newton direction. The Newton direction p_k at x_k is defined by solving

$$\nabla^2 f(x_k) p_k = -\nabla f(x_k),$$

so

$$p_k = -(H^* + E_k)^{-1} (H^* e_k + r_k).$$

Use a first-order inversion (Neumann series). Because H^* is nonsingular and $\|E_k\| = O(\|e_k\|)$ is small for k large, we may expand the inverse:

$$(H^* + E_k)^{-1} = (I + H^{*-1} E_k)^{-1} H^{*-1} = (I - H^{*-1} E_k + O(\|E_k\|^2)) H^{*-1}.$$

Thus

$$(H^* + E_k)^{-1} = H^{*-1} + O(\|e_k\|).$$

Substitute into p_k . Now substitute the expansions:

$$\begin{aligned} p_k &= -\left(H^{*-1} + O(\|e_k\|)\right)\left(H^*e_k + r_k\right) \\ &= -H^{*-1}H^*e_k - H^{*-1}r_k + O(\|e_k\|)H^*e_k + O(\|e_k\|)r_k. \end{aligned}$$

Simplify $H^{*-1}H^* = I$ and use the sizes $\|r_k\| = O(\|e_k\|^2)$, $\|E_k\| = O(\|e_k\|)$:

$$p_k = -e_k + O(\|e_k\|^2) + O(\|e_k\|)O(\|e_k\|^2).$$

Collecting leading orders yields

$$p_k = -(x_k - x^*) + O(\|e_k\|^2).$$

That is,

$$p_k = x^* - x_k + O(\|e_k\|^2).$$

Implication for the Newton update. The Newton update is $x_{k+1} = x_k + p_k$. Using the relation above,

$$x_{k+1} = x_k + (x^* - x_k + O(\|e_k\|^2)) = x^* + O(\|e_k\|^2).$$

Therefore the new error satisfies

$$\|e_{k+1}\| = \|x_{k+1} - x^*\| = O(\|e_k\|^2),$$

which is precisely the quadratic convergence property (for k large enough). This shows that the Newton direction approximates the exact displacement $x^* - x_k$ up to second-order terms, and so Newton's method shrinks the error quadratically once the iterates are sufficiently close to the solution.

Rosenbrock Function: GD vs. Newton – سوال ۱۲

Gradient and Hessian ۱.۷

Compute partial derivatives for $x = (x_1, x_2)^\top$:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix} = \begin{pmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{pmatrix}.$$

The Hessian matrix $H(x) = \nabla^2 f(x)$ is

$$H(x) = \begin{pmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{pmatrix},$$

Implementation ۲.۷

Two methods were implemented in Python (code provided and executed):

1. Steepest Descent (Gradient Descent).

$$x_{k+1} = x_k - \alpha \nabla f(x_k).$$

Two GD variants were run:

- *Fixed step*: $\alpha = 0.001$.
- *Backtracking (Armijo) line search*: starting $\alpha_0 = 1$, shrink factor $\rho = 0.5$, Armijo constant $c = 10^{-4}$.

2. Pure Newton's method.

$$H(x_k) p_k = -\nabla f(x_k), \quad x_{k+1} = x_k + p_k,$$

i.e. full step $\alpha = 1$ (solve linear system for p_k each iteration).

Parameters used ¶.V

- Initial point: $x_0 = [-1.2, 1]^\top$.
- Stopping criterion: $\|\nabla f(x_k)\| < 10^{-6}$ **or** reaching a maximum number of iterations.
- GD fixed: $\alpha = 0.001$, $\text{max_iters} = 50000$.
- GD backtracking: $\alpha_0 = 1$, $\rho = 0.5$, $c = 10^{-4}$, $\text{max_iters} = 2000$.
- Newton: full steps $\alpha = 1$, $\text{max_iters} = 200$.

Numerical results ¶.V

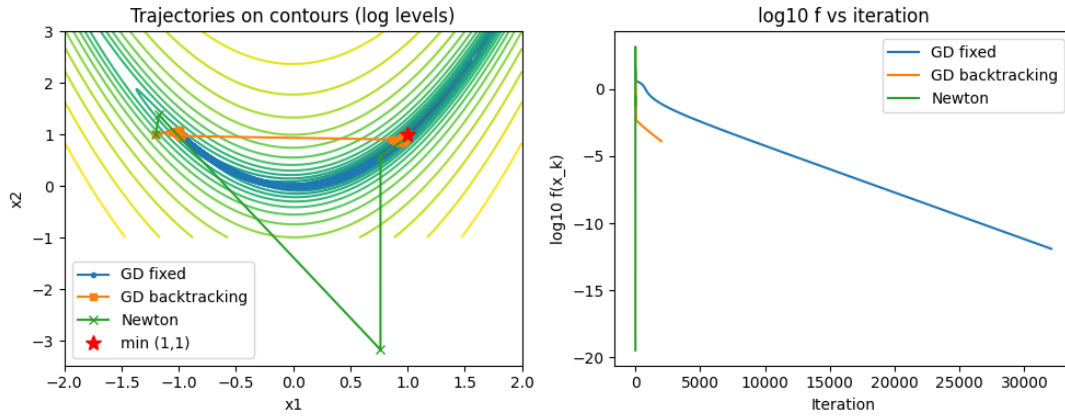
The Python run produced the following numerical outcomes:

Method	iterations used	final $f(x)$	final $\ \nabla f(x)\ $
GD (fixed $\alpha = 0.001$)	32076	1.2517×10^{-12}	9.9989×10^{-7}
GD (backtracking, max 2000 iters)	2000	1.2928×10^{-4}	1.5386×10^{-2}
Newton (full steps)	6	3.4326×10^{-20}	8.2857×10^{-9}

Table 1: Different Approches Numerical Results

- The fixed-step GD reached the gradient tolerance after 32076 iterations and achieved a very small objective value $\sim 10^{-12}$.
- The GD with backtracking was limited by the chosen maximum iterations (2000): after 2000 iterations it had not reached the gradient tolerance and the gradient norm was still $\approx 1.5 \times 10^{-2}$. Its objective was $\approx 1.3 \times 10^{-4}$.
- Newton's method converged in just 6 iterations to machine-level accuracy ($f \approx 10^{-20}$, $\|\nabla f\| \approx 10^{-8}$).

Plots and discussion of convergence rates ٥.٧



شکل ٣: Plots of Convergence rate

Plots produced.

- *Trajectory plot on contour levels:* contours of f (log-scaled levels) with the three method trajectories overlaid. The trajectories show GD (fixed) creeping along the narrow curved valley slowly, GD-backtracking making larger early moves but still following the valley, and Newton quickly jumping toward the valley and rapidly to the minimizer.
- *Log plot $\log_{10} f(x_k)$ vs iteration:* Newton shows a very rapid (superlinear/quadratic) drop in $\log_{10} f$ after a few iterations; GD fixed shows a long, slow decrease (approximately linear behavior in $\log f$ with a small slope); GD-backtracking reduces f faster than tiny fixed steps initially but in this run did not reach the tolerance within the 2000-iteration cap.

Interpretation and convergence behaviour.

1. **Why GD is slow on Rosenbrock.** The Rosenbrock function has a narrow, curved valley (large condition number): one direction (along the valley) is very flat, the orthogonal direction is steep. Plain gradient descent with a small fixed step takes tiny

steps along the valley and therefore needs many iterations to converge. This is reflected in the 32076 iterations for the fixed-step GD (although it eventually reached the tolerance).

2. **Backtracking GD.** Armijo/backtracking often allows larger steps when safe and can dramatically reduce iteration count compared to an overly-small fixed step. In this run the backtracking variant was limited by the imposed $\text{max_iters} = 2000$ and so did not reach the gradient tolerance; with more iterations it would continue decreasing the objective. Backtracking has an extra cost per iteration (several function evaluations) but typically yields more robust step sizes than a fixed tiny α .
3. **Newton's method: fast local convergence.** Newton's method uses curvature information (Hessian) and computes a direction that approximates the exact quadratic model minimizer. When near the solution and with a nonsingular Hessian, Newton's method enjoys *quadratic* convergence: the error roughly squares each iteration. This explains why Newton reached machine accuracy in only 6 iterations for this problem. The contour/trajectory plot typically shows Newton rapidly moving into the curved valley and then converging superlinearly to the minimizer.
4. **Cost per iteration vs iterations trade-off.** Newton iterations are more expensive: evaluate Hessian and solve a 2×2 linear system (cheap here, but expensive in high dimensions). GD iterations are cheap (just a gradient evaluation) but may require many more iterations. The choice between GD and Newton (or quasi-Newton) depends on problem size, cost of Hessian/linear-solve, and desired convergence speed.
5. **Observed convergence rates.** The numerical evidence agrees with theory:
 - GD (fixed small α): roughly linear convergence (slow constant factor), many iterations.
 - GD with backtracking: faster per-iteration reduction than tiny fixed α in many phases, but still fundamentally first-order behavior (linear rate) once near the valley unless step sizes are adjusted carefully.

- Newton: quadratic (superlinear) local convergence - very few iterations to high accuracy.

Step-Size Strategies in Gradient Descent – سوال ١٤

Two Step-Size Selection Methods ١.٨

1. Backtracking and Armijo Line Search. Backtracking (Armijo) line search chooses the step size α dynamically by starting from an initial value (typically $\alpha_0 = 1$) and shrinking it by a factor $\rho \in (0, 1)$ until the *Armijo condition*

$$f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f(x_k)^\top p_k$$

is satisfied. This guarantees sufficient decrease of the objective, avoids overly large steps, and adapts naturally to the geometry of the function.

2. AdaGrad (Adaptive Gradient). AdaGrad rescales each coordinate of the gradient by the inverse square root of the accumulated squared gradients, coordinates with large historical gradients receive smaller effective step sizes. This makes AdaGrad particularly effective on ill-conditioned problems such as Rosenbrock.

Implementation on the Rosenbrock Problem ٢.٨

We consider the Rosenbrock function

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

with starting point

$$x_0 = \begin{bmatrix} -1.2 \\ 1 \end{bmatrix}.$$

We implemented two gradient descent methods:

- GD with Backtracking/Armijo line search.
- GD with AdaGrad (adaptive step sizes).

The numerical results obtained from the provided Python code are:

$$\text{Armijo GD: iterations} = 2000, \quad f(x_{\text{final}}) = 1.29 \times 10^{-4},$$

$$\text{AdaGrad GD: iterations} = 20000, \quad f(x_{\text{final}}) = 5.59 \times 10^{-15}.$$

Final iterates:

$$x_{\text{Armijo}} \approx (0.9887, 0.9774), \quad x_{\text{AdaGrad}} \approx (1.0000, 1.0000).$$

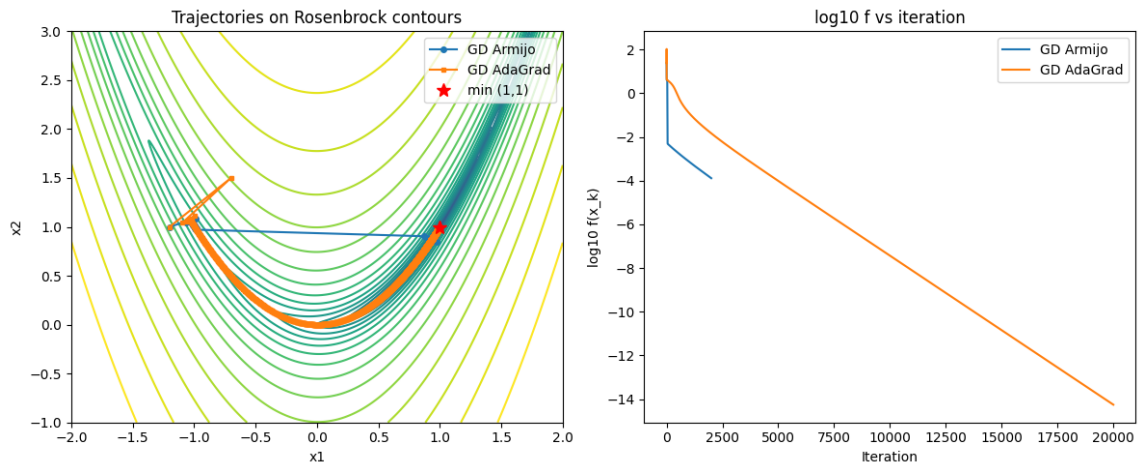
Comparison of Results ۳.۸

The contour plot and trajectory paths show clear differences between the two methods:

- **Armijo GD** quickly makes progress at first but slows down significantly in the narrow curved “valley” of the Rosenbrock function. Due to conservative step-size reductions, it only reaches $f \approx 10^{-4}$ after 2000 iterations. Its trajectory moves almost horizontally before curving toward the minimum.
- **AdaGrad** takes many more iterations overall, but its adaptive per-coordinate scaling makes it excellent at handling the strong anisotropy of the Rosenbrock valley. It steadily follows the curved valley and converges extremely close to the true minimizer: $f \approx 5.6 \times 10^{-15}$.

- **Convergence behavior:** The plot of $\log_{10} f(x_k)$ reveals that Armijo GD decreases rapidly at first, then stagnates. AdaGrad descends more smoothly and continues making progress even in regions where Armijo steps become very small.

Overall, AdaGrad demonstrates superior long-term convergence accuracy on the Rosenbrock function, whereas Backtracking GD is more efficient per iteration but less effective in the ill-conditioned curved valley.



شکل ۴: Trajectories for Armijo GD and AdaGrad

References

- [1] Rosenbrock Function — Optimization Test Problem. Retrieved from: https://en.wikipedia.org/wiki/Rosenbrock_function
- [2] Armijo Rule (Backtracking Line Search). Retrieved from: https://en.wikipedia.org/wiki/Backtracking_line_search
- [3] Bilevel Optimization Overview. Retrieved from: https://en.wikipedia.org/wiki/Bilevel_optimization